

```
{
  "creator" : "ari.neko@cvut.cz",
  "timestamp" : "09-07-2026_17:29:07",
  "variant" : 2,
  "subjectName" : "java-programming",
  "categories" : [ "Generics & Types", "Object-Oriented Principles", "Exception Handling" ],
  "questions" : [ {
    "question" : "What primary objective does Java's Type Erasure accomplish when compiling generic code architectures?",
    "answers" : [ "It replaces type parameters with bounds or Object to ensure type safety without runtime overhead", "It converts object reference addresses into encrypted byte arrays", "It restricts collection instances from scaling horizontally", "It forces the compiler to duplicate class files for every unique type" ],
    "correctIndex" : 0
  }, {
    "question" : "What does the wildcard pattern 'List<? super Integer>' signify inside type boundaries checking?",
    "answers" : [ "It limits the collection strictly to elements that implement serializable", "It specifies a List that can hold Integer or any of its master superclass ancestor layers", "It tells the system that the collection properties are bound to an index", "It prevents downstream functions from executing list mutations" ],
    "correctIndex" : 1
  }, {
    "question" : "What does the wildcard bound pattern 'List<? extends Number>' accomplish inside a method signature?",
    "answers" : [ "It opens the List to accept additions of any custom object structure", "It restricts the parameter to a List containing Number or any of its subclasses", "It mandates that all elements in the List must carry matching static values", "It turns the List into an immutable collection" ],
    "correctIndex" : 1
  }, {
    "question" : "What happens when you use a raw type instead of a parameterized generic type declaration?",
    "answers" : [ "The Java Virtual Machine executes the bytecode lines 50 percent slower", "Jackson's ObjectMapper becomes unable to parse JSON resource file streams", "The application crashes instantly with a ClassCastException upon boot paths", "You lose compile-time type safety checking benefits and trigger compiler warning logs" ],
    "correctIndex" : 3
  }, {
    "question" : "What separates method overriding from method overloading in Java programming execution?",
    "answers" : [ "Overriding modifies class access modifiers; overloading alters return type bounds", "Overloading forces garbage collection routines", "Overriding replaces a superclass method at
```

```

runtime; overloading uses distinct parameters at compile-time", "Overloading is exclusive to
abstract interfaces" ],
"correctIndex" : 2
}, {
"question" : "What restriction applies to a class declared with the final modifier keyword in Java?",
"answers" : [ "The internal variables are rendered immutable values instantly", "The class cannot
be extended or subclassed by any other class structure", "The JVM forces code tasks to run inside
kernel mode", "The class cannot contain any public instance initialization constructors" ],
"correctIndex" : 1
}, {
"question" : "How do you enforce strict Encapsulation inside a Java POJO bean class definition?",
"answers" : [ "Mark the class structure final and avoid using constructors", "Turn all method
trackers static", "Declare fields private and expose access points strictly via public getter and
setter methods", "Route all object initializations through separate database loaders" ],
"correctIndex" : 2
}, {
"question" : "What does a class inherit when it implements a Java Interface containing default
methods?",
"answers" : [ "The interface's default functional implementations without breaking backward
compatibility", "Automated thread synchronization parameters across loop executions", "The
concrete private instance fields", "Complete access to the abstract module's private memory" ],
"correctIndex" : 0
}, {
"question" : "What block in a try-catch-finally layout is guaranteed to execute regardless of
whether an exception is thrown?",
"answers" : [ "The catch block", "The finally block", "The try block", "The throws block" ],
"correctIndex" : 1
}, {
"question" : "What happens if a catch block intercepts an exception and rethrows a new
RuntimeException wrapper?",
"answers" : [ "The current execution path breaks and the exception propagates up the thread
stack frame", "The JVM defaults all text inputs back to empty strings", "The operating system
kernel terminates the parent thread launcher instance", "The active relational database transaction
commits its row status" ],
"correctIndex" : 0
}]
}

```